

highway_rl

March 23, 2026

1 Highway-Env: Conducción Autónoma con RL

1.1 Objetivos de este Notebook

1. **Entender la observación kinematics:** ¿Cómo ve el agente a otros vehículos?
 2. **Estudiar gamma bajo:** ¿Por qué $\gamma=0.8$ funciona mejor que $\gamma=0.99$?
 3. **Transfer learning:** ¿Generaliza de highway a merge?
 4. **Reward shaping:** Velocidad vs seguridad
 5. **Múltiples entornos:** Highway, Parking, Intersection
-

1.2 Prerequisitos

`pip install highway-env stable-baselines3`

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import time

import gymnasium as gym

try:
    import highway_env
    HIGHWAY_AVAILABLE = True
    print("Highway-Env disponible")
except ImportError:
    HIGHWAY_AVAILABLE = False
    print("Instalar con: pip install highway-env")

from stable_baselines3 import DQN, PPO
from stable_baselines3.common.callbacks import BaseCallback
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.monitor import Monitor
```

Instalar con: `pip install highway-env`

2 1. Entornos Disponibles

Entorno	ID	Descripción
Highway	highway-v0	Adelantar coches en autopista
Merge	merge-v0	Incorporarse a autopista
Roundabout	roundabout-v0	Navegar rotonda
Parking	parking-v0	Aparcar en plaza
Intersection	intersection-v0	Cruzar intersección

```
[2]: if HIGHWAY_AVAILABLE:
    env = gym.make("highway-v0")

    print("="*60)
    print("ENTORNO: Highway-v0")
    print("="*60)
    print(f"\nObservación: {env.observation_space}")
    print(f"Acciones: {env.action_space}")

    # Ver observación
    obs, info = env.reset()
    print(f"\nObservación shape: {obs.shape}")
    print(f"Ejemplo: {obs}")

    env.close()
```

3 2. Análisis de la Observación Kinematics

3.1 ¿Cómo ve el agente a otros vehículos?

La observación es una **matriz 5×5** donde: - Fila 0: Ego-vehicle (tu coche) - Filas 1-4: Vehículos cercanos

Cada fila tiene 5 features:

Columna	Feature	Descripción
0	presence	¿Hay vehículo? (0/1)
1	x	Posición X relativa
2	y	Posición Y relativa
3	vx	Velocidad X relativa
4	vy	Velocidad Y relativa

```
[3]: if HIGHWAY_AVAILABLE:
    env = gym.make("highway-v0")
    obs, _ = env.reset()
```

```

print("="*60)
print("OBSERVACIÓN KINEMATICS (5 vehículos × 5 features)")
print("="*60)

features = ["presence", "x", "y", "vx", "vy"]
vehiculos = ["Ego (tú)", "Vehículo 1", "Vehículo 2", "Vehículo 3",
↪ "Vehículo 4"]

print(f"\n{'Vehículo':<15}", end="")
for f in features:
    print(f"{f:>10}", end="")
print()
print("-" * 65)

for i, (nombre, fila) in enumerate(zip(vehiculos, obs)):
    print(f"{nombre:<15}", end="")
    for val in fila:
        print(f"{val:>10.3f}", end="")
    print()

env.close()

```

3.2 Acciones Disponibles

Acción	Descripción
0	LANE_LEFT (cambiar carril izquierda)
1	IDLE (mantener)
2	LANE_RIGHT (cambiar carril derecha)
3	FASTER (acelerar)
4	SLOWER (frenar)

3.3 Función de Recompensa

Evento	Recompensa
Colisión	-1
Velocidad alta	+0.4 (proporcional)
Carril derecho	+0.1
Cambio de carril	-0.1 (pequeña penalización)

4 3. Código Base

```
[4]: class HighwayCallback(BaseCallback):
    """Callback para highway-env."""

    def __init__(self, verbose=0):
        super().__init__(verbose)
        self.episode_rewards = []
        self.crashes = []

    def _on_step(self) -> bool:
        for info in self.locals.get('infos', []):
            if 'episode' in info:
                self.episode_rewards.append(info['episode']['r'])
            if 'crashed' in info:
                self.crashes.append(info['crashed'])
        return True

def entrenar_highway(env_id="highway-v0", timesteps=30000, algoritmo="DQN",
    ↪**kwargs):
    """Entrena en un entorno de highway-env."""
    env = gym.make(env_id)
    env = Monitor(env)

    config = {
        "learning_rate": 5e-4,
        "gamma": 0.8, # Horizonte corto para conducción
        "buffer_size": 50000,
    }
    config.update(kwargs)

    if algoritmo == "DQN":
        model = DQN("MlpPolicy", env, verbose=0, **config)
    else:
        model = PPO("MlpPolicy", env, verbose=0,
            learning_rate=config['learning_rate'],
            gamma=config['gamma'])

    callback = HighwayCallback()
    model.learn(total_timesteps=timesteps, callback=callback, progress_bar=True)

    env.close()
    return model, callback

print("Funciones cargadas")
```

Funciones cargadas

5 4. VARIANTE A: Transfer Learning (Highway → Merge)

```
[5]: def estudiar_transfer_learning(timesteps=20000):  
    """  
    Estudia si un modelo entrenado en highway generaliza a merge.  
    """  
    if not HIGHWAY_AVAILABLE:  
        return {}  
  
    print("="*60)  
    print("VARIANTE A: Transfer Learning")  
    print("="*60)  
  
    resultados = {}  
  
    # 1. Entrenar en highway  
    print("\n1. Entrenando en Highway...")  
    model_highway, cb_highway = entrenar_highway("highway-v0", timesteps)  
  
    # Evaluar en highway  
    env_eval = gym.make("highway-v0")  
    mean_hw, std_hw = evaluate_policy(model_highway, env_eval,   
↪n_eval_episodes=10)  
    env_eval.close()  
    print(f"    En Highway: {mean_hw:.1f} ± {std_hw:.1f}")  
  
    # 2. Evaluar en merge (sin reentrenar)  
    print("\n2. Evaluando en Merge (sin reentrenar)...")  
    env_merge = gym.make("merge-v0")  
    mean_merge_transfer, std_merge = evaluate_policy(model_highway, env_merge,   
↪n_eval_episodes=10)  
    env_merge.close()  
    print(f"    Transfer a Merge: {mean_merge_transfer:.1f} ± {std_merge:.1f}")  
  
    # 3. Entrenar desde cero en merge  
    print("\n3. Entrenando desde cero en Merge...")  
    model_merge, cb_merge = entrenar_highway("merge-v0", timesteps)  
  
    env_merge2 = gym.make("merge-v0")  
    mean_merge_scratch, _ = evaluate_policy(model_merge, env_merge2,   
↪n_eval_episodes=10)  
    env_merge2.close()  
    print(f"    Desde cero en Merge: {mean_merge_scratch:.1f}")
```

```

resultados = {
    'highway_rewards': cb_highway.episode_rewards,
    'highway_eval': mean_hw,
    'transfer_to_merge': mean_merge_transfer,
    'merge_from_scratch': mean_merge_scratch
}

# Conclusión
print("\n" + "-"*60)
print("CONCLUSIÓN:")
if mean_merge_transfer > mean_merge_scratch * 0.5:
    print(" El modelo SÍ transfiere conocimiento a merge")
else:
    print(" El modelo NO transfiere bien a merge")

return resultados

if HIGHWAY_AVAILABLE:
    resultados_a = estudiar_transfer_learning(timesteps=15000)

```

6 5. VARIANTE B: Estudio de Gamma

6.0.1 ¿Por qué gamma bajo para conducción?

En conducción autónoma: - Las decisiones deben ser **rápidas y locales** - El futuro lejano es **muy incierto** (otros conductores) - $\gamma=0.8$ funciona mejor que $\gamma=0.99$

```

[6]: def estudiar_gamma_highway(timesteps=15000):
    """
    Compara diferentes valores de gamma en highway.
    """
    if not HIGHWAY_AVAILABLE:
        return {}

    print("="*60)
    print("VARIANTE B: Estudio de Gamma en Highway")
    print("="*60)

    gammas = [0.8, 0.9, 0.95, 0.99]
    resultados = {}

    for gamma in gammas:
        print(f"\nEntrenando con gamma={gamma}...")
        model, callback = entrenar_highway("highway-v0", timesteps, gamma=gamma)

        env_eval = gym.make("highway-v0")

```

```

        mean_reward, std_reward = evaluate_policy(model, env_eval,
↪n_eval_episodes=10)
        env_eval.close()

        resultados[gamma] = {
            'rewards': callback.episode_rewards,
            'mean': mean_reward,
            'std': std_reward
        }
        print(f"  gamma={gamma}: {mean_reward:.1f} ± {std_reward:.1f}")

    return resultados

if HIGHWAY_AVAILABLE:
    resultados_b = estudiar_gamma_highway(timesteps=10000)

```

```

[7]: # Visualizar
if HIGHWAY_AVAILABLE and 'resultados_b' in dir() and resultados_b:
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    for gamma, data in resultados_b.items():
        rewards = data['rewards']
        if len(rewards) > 5:
            smoothed = np.convolve(rewards, np.ones(5)/5, mode='valid')
            axes[0].plot(smoothed, label=f'={gamma}')
    axes[0].set_xlabel('Episodio')
    axes[0].set_ylabel('Recompensa')
    axes[0].set_title('Efecto de Gamma en Highway')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    gammas = list(resultados_b.keys())
    means = [resultados_b[g]['mean'] for g in gammas]
    axes[1].bar([str(g) for g in gammas], means)
    axes[1].set_xlabel('Gamma')
    axes[1].set_ylabel('Recompensa Final')
    axes[1].set_title('Rendimiento por Gamma')

    plt.tight_layout()
    plt.show()

```

7 6. VARIANTE C: Reward Shaping (Velocidad vs Seguridad)

```
[8]: def crear_entorno_custom(reward_speed_range=[0.2, 0.5], collision_reward=-1.0):  
    """  
    Crea entorno con recompensa personalizada.  
    """  
    env = gym.make("highway-v0")  
  
    # Configurar recompensa  
    env.unwrapped.config["reward_speed_range"] = reward_speed_range  
    env.unwrapped.config["collision_reward"] = collision_reward  
  
    return Monitor(env)  
  
def estudiar_reward_shaping(timesteps=15000):  
    """  
    Compara diferentes configuraciones de recompensa.  
    """  
    if not HIGHWAY_AVAILABLE:  
        return {}  
  
    print("="*60)  
    print("VARIANTE C: Reward Shaping")  
    print("="*60)  
  
    configs = {  
        "Balanceado": {"reward_speed_range": [0.2, 0.5], "collision_reward": -1.  
↪ 0},  
        "Solo Velocidad": {"reward_speed_range": [0.5, 1.0], "collision_reward":  
↪ -0.5},  
        "Solo Seguridad": {"reward_speed_range": [0.0, 0.2], "collision_reward":  
↪ -5.0},  
    }  
  
    resultados = {}  
  
    for nombre, config in configs.items():  
        print(f"\nEntrenando '{nombre}'...")  
        print(f"    speed_range={config['reward_speed_range']},  
↪ collision={config['collision_reward']}")  
  
        env = crear_entorno_custom(**config)  
  
        model = DQN("MlpPolicy", env, verbose=0,  
                    learning_rate=5e-4, gamma=0.8, buffer_size=50000)
```



```

        callback = HighwayCallback()
        model.learn(total_timesteps=timesteps, callback=callback,
↪progress_bar=True)

        # Evaluar
        env_eval = gym.make("highway-v0")
        mean_reward, std_reward = evaluate_policy(model, env_eval,
↪n_eval_episodes=10)

        resultados[nombre] = {
            'rewards': callback.episode_rewards,
            'crashes': callback.crashes,
            'mean': mean_reward,
            'std': std_reward
        }

        crash_rate = np.mean(callback.crashes) if callback.crashes else 0
        print(f" Resultado: {mean_reward:.1f} (crash rate: {crash_rate:.2%})")

        env.close()
        env_eval.close()

    return resultados

if HIGHWAY_AVAILABLE:
    resultados_c = estudiar_reward_shaping(timesteps=10000)

```

8 7. VARIANTE D: DQN vs PPO en Parking

```

[9]: def comparar_en_parking(timesteps=20000):
    """
    Compara DQN y PPO en el entorno de parking.
    """
    if not HIGHWAY_AVAILABLE:
        return {}

    print("="*60)
    print("VARIANTE D: DQN vs PPO en Parking")
    print("="*60)

    resultados = {}

    for algo in ["DQN", "PPO"]:
        print(f"\nEntrenando {algo} en parking...")

```

```

env = gym.make("parking-v0")
env = Monitor(env)

if algo == "DQN":
    model = DQN("MlpPolicy", env, verbose=0,
                learning_rate=5e-4, gamma=0.8, buffer_size=50000)
else:
    model = PPO("MlpPolicy", env, verbose=0,
                learning_rate=5e-4, gamma=0.8)

callback = HighwayCallback()
model.learn(total_timesteps=timesteps, callback=callback,
            ↪progress_bar=True)

env_eval = gym.make("parking-v0")
mean_reward, std_reward = evaluate_policy(model, env_eval,
            ↪n_eval_episodes=10)
env_eval.close()

resultados[algo] = {
    'rewards': callback.episode_rewards,
    'mean': mean_reward,
    'std': std_reward
}

print(f" {algo}: {mean_reward:.1f} ± {std_reward:.1f}")
env.close()

return resultados

if HIGHWAY_AVAILABLE:
    resultados_d = comparar_en_parking(timesteps=10000)

```

9 8. Conclusiones

9.1 ¿Qué aprendimos?

1. **Observación kinematics:** Matriz 5×5 con información relativa de vehículos cercanos
2. **Gamma bajo funciona mejor:** $\gamma=0.8$ es mejor que $\gamma=0.99$ para conducción porque:
 - Las decisiones son locales
 - El futuro es muy incierto
3. **Transfer learning:** El modelo puede generalizar parcialmente entre entornos similares
4. **Reward shaping:** El balance velocidad/seguridad es crucial

9.2 Referencias

- [Highway-Env Documentation](#)
- [RL for Autonomous Driving Survey](#)

9.3 Variantes de Entrenamiento — Highway

Las variantes en Highway-Env exploran una pregunta diferente: **¿cómo transferir conocimiento entre entornos?**

Con 7 entornos disponibles (autopista, rotonda, intersección...), podemos entrenar un agente en uno y ver cómo se adapta a otros.

Variante	Estrategia	Entornos	Concepto
A	Entorno único (<i>actual</i>)	1 entorno	Baseline
B	Transfer Learning	2 entornos	Conocimiento reutilizable
C	Curriculum Learning	5 entornos prog.	Aprender gradualmente

Entornos disponibles (orden de dificultad aproximado): highway-fast → highway → merge → roundabout → intersection

9.3.1 Variante A — Entrenamiento en Entorno Único (*actual*)

```
python highway_conduccion.py --env highway --algorithm DQN
```

```
python highway_conduccion.py --env intersection --algorithm DQN
```

Entrena DQN o PPO en un entorno específico. Es el baseline para comparar con las otras variantes.

Observación: matriz 5×5 de cinemática (5 vehículos × 5 características: presencia, x, y, vx, vy)

Acciones: LANE_LEFT, IDLE, LANE_RIGHT, FASTER, SLOWER **Reward:** velocidad alta (+0.4), carril derecho (+0.1), colisión (-1.0)

```
[10]: # Variante A: Entrenamiento en un solo entorno
# from highway_conduccion import entrenar_highway
# model, callback = entrenar_highway(env_name="highway", timesteps=50000,
# ↪ algorithm="DQN")

print("Variante A: Entrenamiento en entorno único")
print()
print("Entornos disponibles:")
entornos = {
    "highway": "Autopista: adelantar coches, mantener velocidad",
    "highway-fast": "Autopista simplificada (más rápida)",
    "merge": "Incorporarse a autopista",
    "roundabout": "Rotonda",
    "parking": "Aparcar en plaza (acciones continuas)",
    "intersection": "Cruzar intersección",
    "racetrack": "Circuito de carreras",
```

```

}
for name, desc in entornos.items():
    print(f"  {name:<15}: {desc}")
print()
print("Para entrenar en cualquier entorno:")
print("  entrenar_highway(env_name='intersection', timesteps=50000,
    ↪algorithm='DQN')")

```

Variante A: Entrenamiento en entorno único

Entornos disponibles:

highway	: Autopista: adelantar coches, mantener velocidad
highway-fast	: Autopista simplificada (más rápida)
merge	: Incorporarse a autopista
roundabout	: Rotonda
parking	: Aparcar en plaza (acciones continuas)
intersection	: Cruzar intersección
racetrack	: Circuito de carreras

Para entrenar en cualquier entorno:

```
entrenar_highway(env_name='intersection', timesteps=50000, algorithm='DQN')
```

9.3.2 Variante B — Transfer Learning entre Entornos

```
python highway_conduccion.py --transfer
```

```
python highway_conduccion.py --transfer --source highway --target intersection
```

Idea: las habilidades aprendidas en autopista (mantener velocidad, no chocar) son parcialmente útiles al cruzar una intersección, aunque el escenario sea diferente.

Flujo: 1. **Fase 1:** Entrenar en entorno *fuentes* (más sencillo, ej. highway) hasta convergencia 2.

Fase 2: Transferir pesos al entorno *objetivo* (más complejo, ej. intersection) y hacer fine-tuning

```
highway (simple)      entrenar      pesos red
```

```
set_env(intersection)
```

```
intersection (complejo)  fine-tune  política final
```

Hiperparámetro crítico: learning rate del fine-tuning. Debe ser menor que el entrenamiento base para no “olvidar” lo aprendido (*catastrophic forgetting*).

```

[11]: # Variante B: Transfer Learning
# from highway_conduccion import transfer_learning
# model, cb_src, cb_tgt = transfer_learning(
#     source_env="highway",
#     target_env="intersection",
#     source_timesteps=80000,
#     target_timesteps=40000,
#     algorithm="DQN"

```

```

# )
# Genera: highway_transfer_learning.png

print("Variante B: Transfer Learning highway → intersection")
print()
transfer_code = """
# Fase 1: entrenar en fuente
model, cb_source = entrenar_highway("highway", timesteps=80000, algorithm="DQN")

# Fase 2: transferir al objetivo
env_target = crear_entorno("intersection")
model.set_env(env_target)          # Mismo modelo, nuevo entorno
model.learning_rate = 1e-4         # LR reducido: fine-tuning suave

model.learn(
    total_timesteps=40000,
    reset_num_timesteps=False,      # No reiniciar el contador
)
"""
print(transfer_code)
print("La función también entrena un modelo desde cero en el objetivo")
print("para comparar 'Con transfer' vs 'Sin transfer'.")

```

Variante B: Transfer Learning highway → intersection

```

# Fase 1: entrenar en fuente
model, cb_source = entrenar_highway("highway", timesteps=80000, algorithm="DQN")

# Fase 2: transferir al objetivo
env_target = crear_entorno("intersection")
model.set_env(env_target)          # Mismo modelo, nuevo entorno
model.learning_rate = 1e-4         # LR reducido: fine-tuning suave

model.learn(
    total_timesteps=40000,
    reset_num_timesteps=False,      # No reiniciar el contador
)

```

La función también entrena un modelo desde cero en el objetivo para comparar 'Con transfer' vs 'Sin transfer'.

9.3.3 Variante C — Curriculum Learning Progresivo

`python highway_conduccion.py --curriculum`

El agente aprende conducción de forma gradual, como un estudiante humano:

Nivel	Entorno	Habilidad añadida
1	highway-fast	Flujo básico, no chocar
2	highway	Adelantamientos, cambios de carril
3	merge	Incorporación, gestión de huecos
4	roundabout	Decisiones de giro, prioridad
5	intersection	Coordinación compleja

Por qué funciona: cada nivel añade una habilidad nueva sobre las anteriores. El agente no tiene que aprender todo desde cero al cambiar de entorno.

Comparación con Variante A (entrenamiento directo en intersection): - Sin curriculum: el agente ve colisiones constantemente al principio → señal de reward muy escasa - Con curriculum: el agente ya sabe conducir cuando llega a intersection → aprende más rápido

```
[12]: # Variante C: Curriculum Learning
# from highway_conduccion import curriculum_learning
# model, historial = curriculum_learning(timesteps_per_env=20000,
# ↪ algorithm="DQN")
# Genera: highway_curriculum.png

print("Variante C: Curriculum Learning (5 niveles)")
print()
curriculum_code = """
curriculum = [
    ("highway-fast", "Autopista simple"),      # Nivel 1
    ("highway",      "Autopista completa"),    # Nivel 2
    ("merge",        "Incorporación"),          # Nivel 3
    ("roundabout",   "Rotonda"),                # Nivel 4
    ("intersection", "Intersección"),          # Nivel 5
]

# En cada nivel:
if nivel == 0:
    model = DQN("MlpPolicy", env, ...) # Crear modelo nuevo
else:
    model.set_env(env)                  # Transferir al siguiente entorno
    model.learning_rate *= 0.8          # Reducir LR progresivamente

model.learn(timesteps_per_env, reset_num_timesteps=(nivel==0))
model.save(f"nivel_{nivel}_{env_name}.zip") # Checkpoint por nivel
"""
print(curriculum_code)
```

Variante C: Curriculum Learning (5 niveles)

```
curriculum = [
```

```

    ("highway-fast", "Autopista simple"),      # Nivel 1
    ("highway",      "Autopista completa"),    # Nivel 2
    ("merge",        "Incorporación"),         # Nivel 3
    ("roundabout",   "Rotonda"),               # Nivel 4
    ("intersection", "Intersección"),          # Nivel 5
]

# En cada nivel:
if nivel == 0:
    model = DQN("MlpPolicy", env, ...) # Crear modelo nuevo
else:
    model.set_env(env)                  # Transferir al siguiente entorno
    model.learning_rate *= 0.8          # Reducir LR progresivamente

model.learn(timesteps_per_env, reset_num_timesteps=(nivel==0))
model.save(f"nivel_{nivel}_{env_name}.zip") # Checkpoint por nivel

```

9.3.4 Comparativa de Variantes

Aspecto	A: Único	B: Transfer	C: Curriculum
Entornos	1	2	5
Complejidad impl.	Baja	Media	Media
Rendimiento en entorno fácil	Alto	Alto	Alto
Rendimiento en entorno difícil	Bajo	Medio	Alto
Tiempo total	Corto	Medio	Largo

Cuándo usar cada variante: - **A** — Baseline o cuando solo interesa un entorno - **B** — Cuando tienes un entorno fuente y quieres adaptar a otro - **C** — Cuando quieres el mejor rendimiento posible en el entorno más difícil

Lección: el curriculum learning es una de las técnicas más potentes para entornos difíciles donde el reward es escaso al principio.